

Atividade Prática – Aula 3

Modelagem com e sem Estrutura no MongoDB

Objetivo e trabalhar:

- Flexibilidade
- Controle estrutural
- Comparação prática
- Raciocínio crítico

EXERCÍCIO 1 – Banco Sem Estrutura Definida

(BD_SE – Banco de Dados Sem Estrutura)

Contexto:

Uma startup de games está começando e quer registrar:

- Jogadores
- Campeonatos

Mas ainda não definiu regras rígidas de estrutura.

Passo 1 – Criar o banco

```
use BD_SE
```

Passo 2 – Criar Coleção Jogadores (sem schema)

```
db.jogadores.insertMany([
  { nome: "Lucas", nick: "ProPlayer", ranking: 1500 },
  { nome: "Ana", idade: 22 },
  { nome: "Carlos", nick: "Sniper", patrocinador: "RedForce" }
])
```

Observe:

- Documentos com estruturas diferentes
- Campos ausentes
- Campos extras

Passo 3 – Criar Coleção Campeonatos

```
db.campeonatos.insertMany([
  { nome: "Copa Brasil", premio: 50000 },
  { nome: "Liga Sudeste", data: "2025-06-01" },
  { nome: "Desafio CE", premio: 20000, online: true }
])
```

Reflexão

- Todos os documentos seguem um padrão?
- Isso pode gerar problema em sistemas grandes?
- Como o sistema sabe o que é obrigatório?

Aqui nasce a necessidade do schema.

EXERCÍCIO 2 – Banco Com Estrutura Definida**(BD_CE – Banco de Dados Com Estrutura)**

Agora o mesmo modelo, mas com controle.

♦ Tabela de Tipos BSON

Tipo MongoDB	Equivalente PostgreSQL	Exemplo
string	VARCHAR / TEXT	"Lucas"
int	INTEGER	25
double	DECIMAL	199.99
bool	BOOLEAN	true
date	DATE / TIMESTAMP	ISODate("2025-06-01")
array	ARRAY	["FPS", "MOBA"]
object	JSON / RECORD	{ cidade: "SP" }

Passo 1 – Criar Banco

```
use BD_CE
```

Passo 2 – Criar Coleção Jogadores com Schema

```
db.createCollection("jogadores", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nome", "nick", "ranking"],
      properties: {
        nome: {
          bsonType: "string",
          description: "Nome deve ser string e é obrigatório"
        },
        nick: {
          bsonType: "string"
        },
        ranking: {
          bsonType: "int",
          minimum: 0
        },
        patrocinador: {
          bsonType: "string"
        }
      }
    }
  }
})
```

Passo 3 – Criar Coleção Campeonatos com Schema

```
db.createCollection("campeonatos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nome", "premio"],
      properties: {
        nome: {
          bsonType: "string"
        },
        premio: {
```

```
    bsonType: "double"
  },
  data: {
    bsonType: "date"
  },
  online: {
    bsonType: "bool"
  }
}
}
```

Teste de Inserção Válida

```
db.jogadores.insertOne({
  nome: "Lucas",
  nick: "ProPlayer",
  ranking: 1500
})
```

Funciona

Teste de Inserção Inválida

```
db.jogadores.insertOne({
  nome: "Ana"
})
```

Erro



Porque campos obrigatórios estão ausentes.

Comparação Final para Encerrar a Aula

BD_SE	BD_CE
Flexível	Controlado
Estrutura variável	Estrutura validada
Mais rápido para prototipar	Mais seguro para produção
Pode gerar inconsistência	Mantém padrão

Fechamento

MongoDB não é desorganizado.

Ele é flexível por padrão, mas pode ser estruturado quando necessário.

ATIVIDADE AVALIATIVA – Aula 3

Modelagem no MongoDB: Flexibilidade x Estrutura

Entrega: Script, prints e respostas discursivas curtas

Contexto da Atividade

Você foi contratado por uma empresa de e-sports que está iniciando seu sistema em MongoDB.

A equipe criou dois bancos:

- **BD_SE:** Banco sem validação de estrutura
- **BD_CE:** Banco com validação de schema

Seu objetivo é analisar, testar e comparar os dois modelos.

PARTE 1 – Banco Sem Estrutura (BD_SE)

Tarefa 1 – Inserção Livre

Insira:

- Um jogador com apenas nome
- Um jogador com nome, ranking e patrocinador
- Um jogador com um campo inesperado chamado “redeSocial”

O que deve acontecer?

Todos devem ser inseridos com sucesso.

O aluno deve apresentar o comando e o print.

Tarefa 2 – Análise Crítica

Responda:

1. Todos os documentos possuem o mesmo padrão?
 2. Isso pode gerar problemas em consultas futuras?
 3. Cite um risco desse modelo em sistemas grandes.
-

Aqui se avalia compreensão, não só execução.

PARTE 2 – Banco Com Estrutura (BD_CE)

Tarefa 3 – Inserção Correta

Insira um jogador válido, obedecendo o schema.

Deve funcionar.

Tarefa 4 – Inserção Inválida

Tente inserir um jogador:

- Sem campo obrigatório
OU
- Com ranking como string

Deve gerar erro.

Você deve mostrar o erro no print.

Tarefa 5 – Comparação Técnica

Responda:

1. Qual banco oferece mais segurança para produção?
 2. Qual banco oferece mais flexibilidade para prototipação?
 3. Em qual cenário você utilizaria cada modelo?
-

PARTE 3 – Raciocínio de Arquitetura

Situação-problema (estilo ENADE):

Uma empresa de jogos está crescendo rapidamente e pretende integrar seu banco com uma API pública.

O sistema será acessado por milhares de usuários simultaneamente.

Com base nisso:

- Você recomendaria BD_SE ou BD_CE?
- Justifique tecnicamente.

Hoje vocês aprenderam que MongoDB pode ser flexível, mas banco de dados não é lugar de improviso. Estrutura é decisão estratégica.

Entrega da Atividade

1. Exporte/salve o arquivo de acordo com o padrão: **BDN-Atividade-Aula03.pdf**
 2. Enviar a Atividade em PDF no GitHub:
 1. Acesse seu repositório da disciplina: **FATEC-JCR-3DSM-BDN-2026-1-seunome**
 2. Se ainda não existir, você deverá criar a pasta da atividade: **P1-Conteudos/Atividades/Atividade-Aula03/README.md**
 3. Faça o **commit** do pdf da atividade feita na pasta.
 4. Em seguida copie o **link** do pdf.
 5. Acesse o **Kanban-BDN-2026-1** da disciplina que se encontra no repositório da professora.
 6. Clique no card: **BDN-Atividade-Aula03**.
 7. No comentário, cole o **link** do **pdf** da atividade que você fez.
 8. Volte ao seu repositório e acesse o seu **Kanban-BDN-2026-1**.
 9. Mova o card **BDN-Atividade-Aula03** para a coluna **Entregue**.
-